

K means

Enlace colab:

https://colab.research.google.com/drive/1d_5qvvhdbkNB-LZ1lwGqC4cjv39NfsrX?usp=sharing

Origina scatter

[54]

✓ 1s

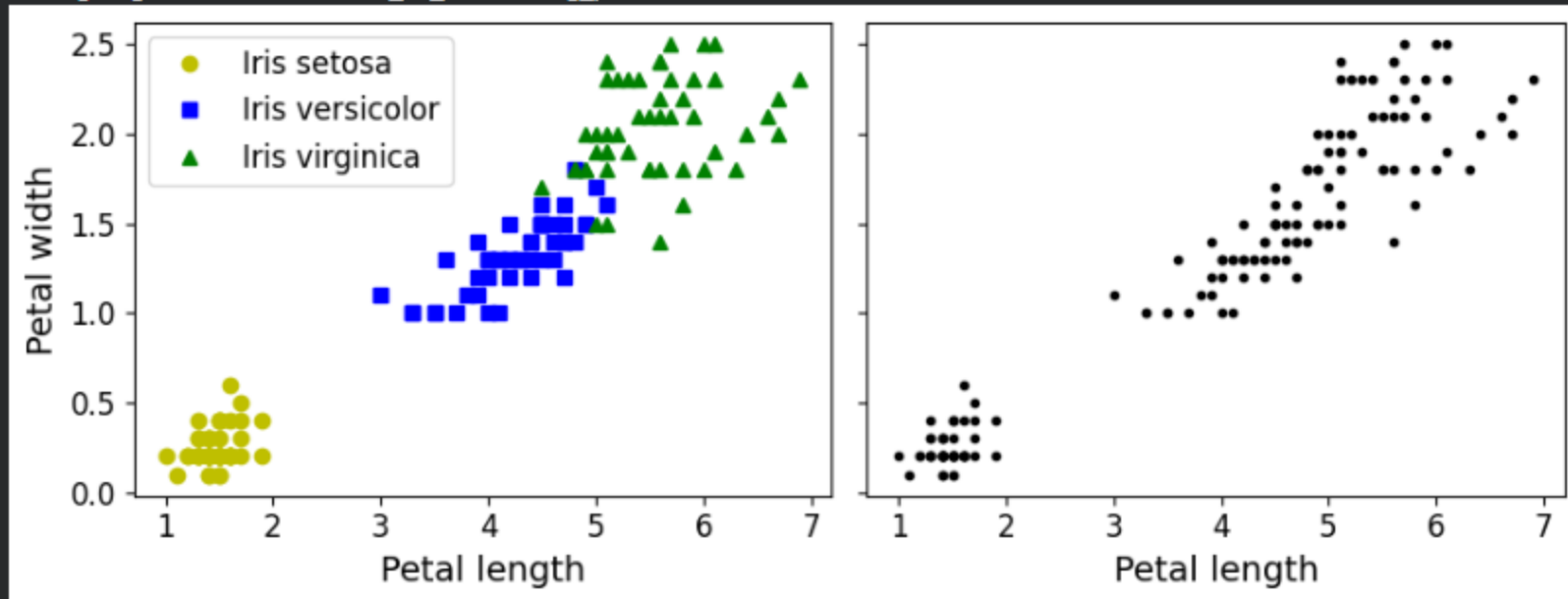
```
plt.figure(figsize=(9, 3.5))

plt.subplot(121)
plt.plot(X[y==0, 2], X[y==0, 3], "yo", label="Iris setosa")
plt.plot(X[y==1, 2], X[y==1, 3], "bs", label="Iris versicolor")
plt.plot(X[y==2, 2], X[y==2, 3], "g^", label="Iris virginica")
plt.xlabel("Petal length", fontsize=14)
plt.ylabel("Petal width", fontsize=14)
plt.legend(fontsize=12)

plt.subplot(122)
plt.scatter(X[:, 2], X[:, 3], c="k", marker=".")
plt.xlabel("Petal length", fontsize=14)
plt.tick_params(labelleft=False)

save_fig("classification_vs_clustering_plot")
plt.show()
```

Saving figure classification_vs_clustering_plot



✓ K-Means

Let's start by generating some blobs:

Modificado scatter

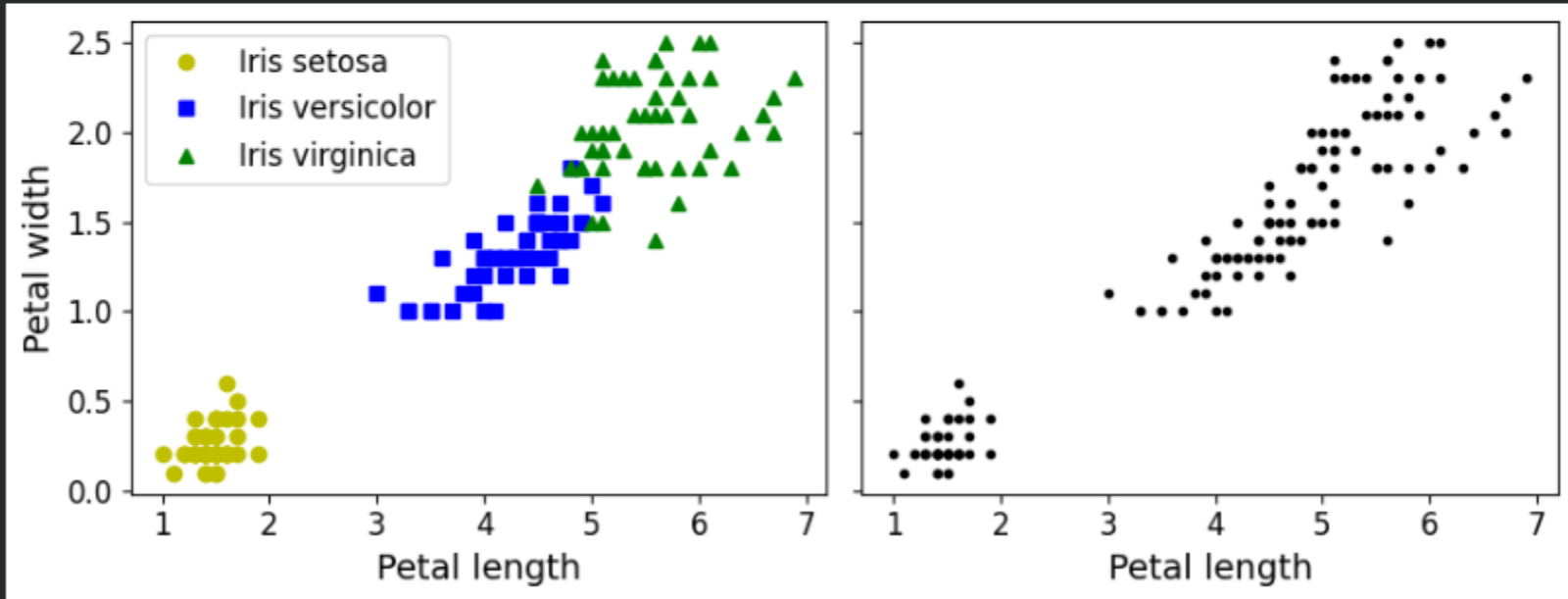
```
[426]
✓ 0s ▶ plt.figure(figsize=(9, 3.5))

plt.subplot(121)
plt.plot(X[y==0, 2], X[y==0, 3], "yo", label="Iris setosa")
plt.plot(X[y==1, 2], X[y==1, 3], "bs", label="Iris versicolor")
plt.plot(X[y==2, 2], X[y==2, 3], "g^", label="Iris virginica")
plt.xlabel("Petal length", fontsize=14)
plt.ylabel("Petal width", fontsize=14)
plt.legend(fontsize=12)

plt.subplot(122)
plt.scatter(X[:, 2], X[:, 3], c="k", marker=".")
plt.xlabel("Petal length", fontsize=14)
plt.tick_params(labelleft=False)

save_fig("classification_vs_clustering_plot")
plt.show()
```

... Saving figure classification_vs_clustering_plot



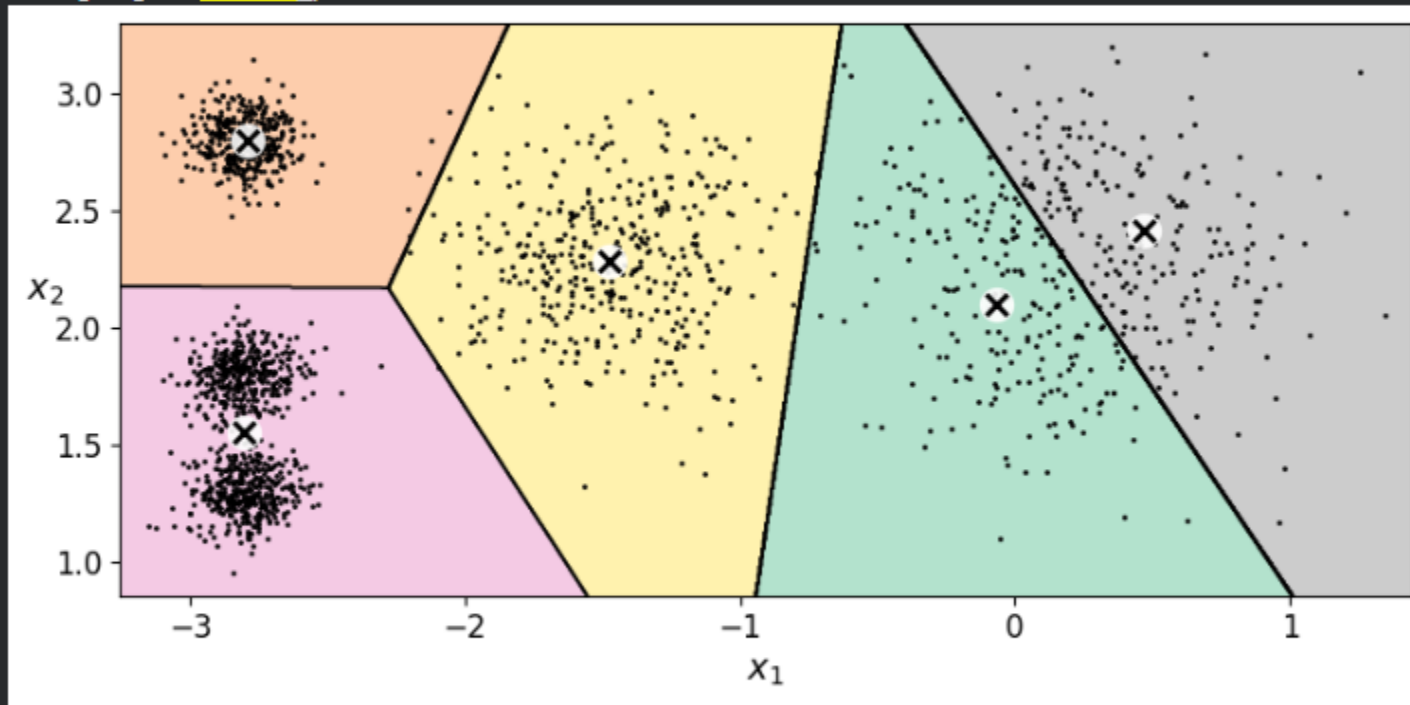
Voronoi Original

[69]

✓ 2 s

```
plt.figure(figsize=(8, 4))  
plot_decision_boundaries(kmeans, X)  
save_fig("voronoi_plot")  
plt.show()
```

▼ Saving figure **voronoi_plot**



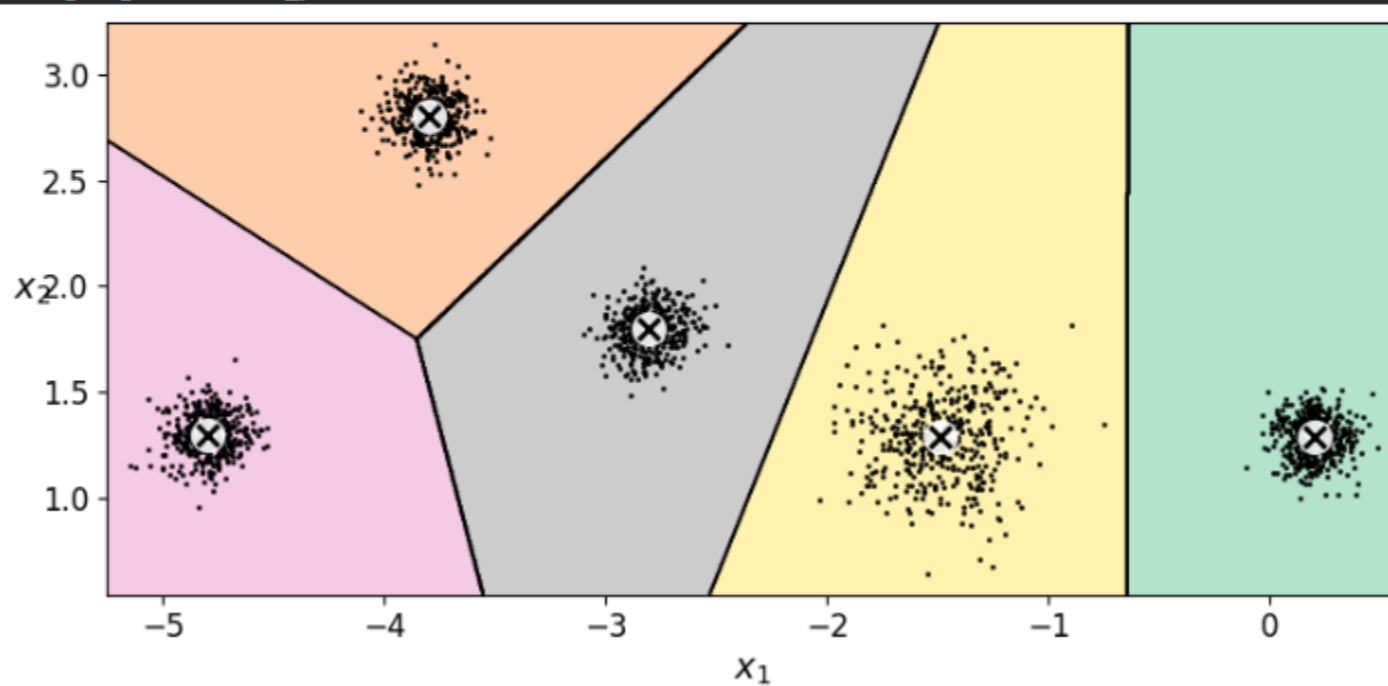
▼ The K-Means Algorithm

Voronoi modificado

```
plt.ylabel("$x_2$", fontsize=14, rotation=0)  
else:  
    plt.tick_params(labelleft=False)
```

```
plt.figure(figsize=(8, 4))  
plot_decision_boundaries(kmeans, X)  
save_fig("voronoi_plot")  
plt.show()
```

Saving figure voronoi_plot

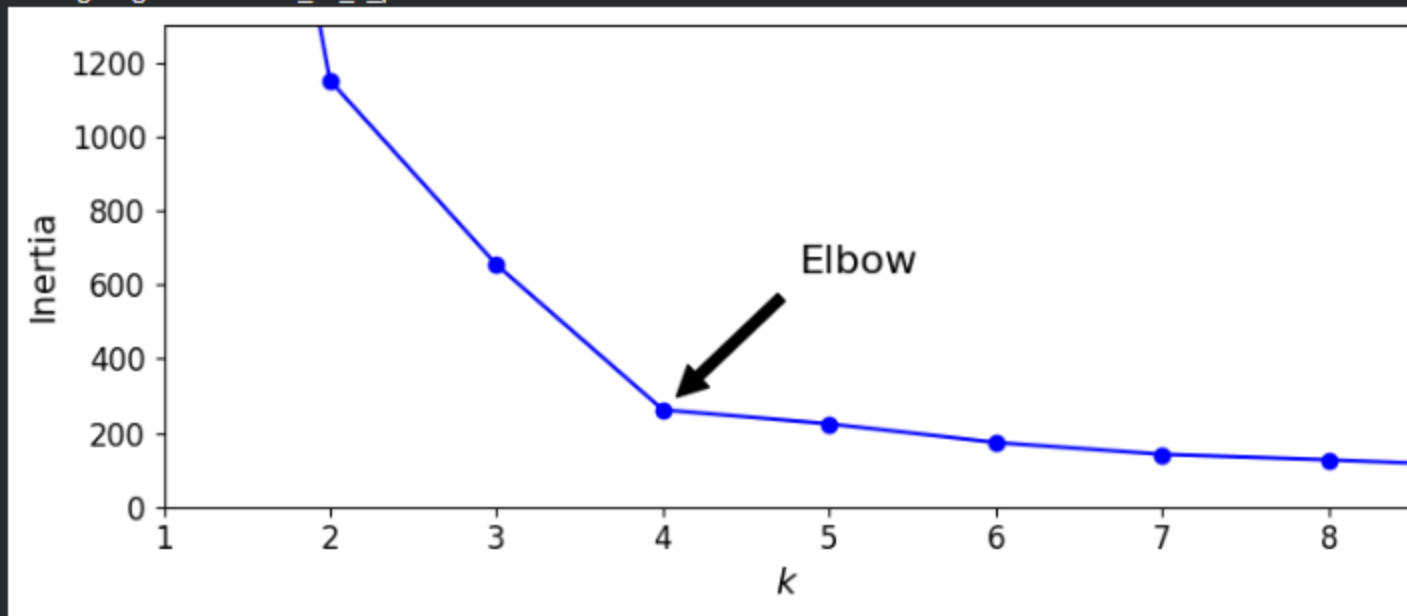


Codo original

```
[88] ✓ 0s kmeans_per_k = [KMeans(n_clusters=k, random_state=42).fit(X)
                  for k in range(1, 10)]
inertias = [model.inertia_ for model in kmeans_per_k]
```

```
[89] ✓ 0s plt.figure(figsize=(8, 3.5))
plt.plot(range(1, 10), inertias, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Inertia", fontsize=14)
plt.annotate('Elbow',
             xy=(4, inertias[3]),
             xytext=(0.55, 0.55),
             textcoords='figure fraction',
             fontsize=16,
             arrowprops=dict(facecolor='black', shrink=0.1)
            )
plt.axis([1, 8.5, 0, 1300])
save_fig("inertia_vs_k_plot")
plt.show()
```

*** Saving figure inertia_vs_k_plot

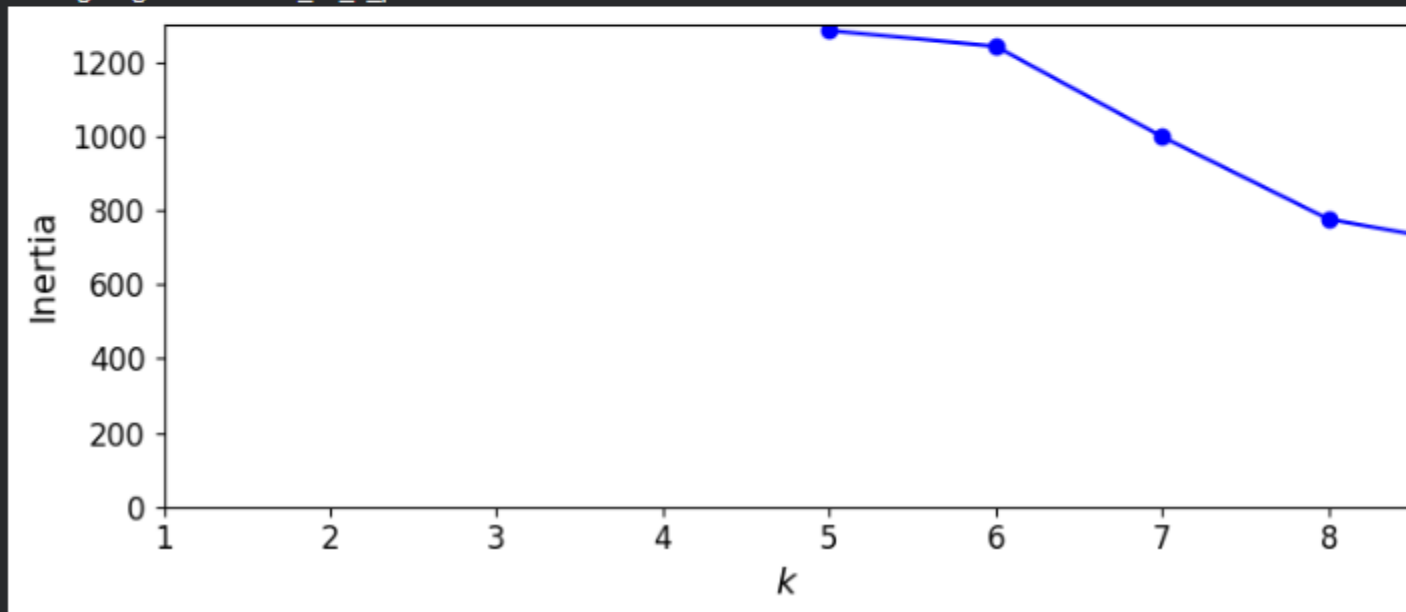


Codo modificado

```
[410]
✓ 0s kmeans_per_k = [KMeans(n_clusters=k, random_state=42).fit(X)
                  for k in range(1, 10)]
inertias = [model.inertia_ for model in kmeans_per_k]
```

```
[411]
✓ 0s ▶ plt.figure(figsize=(8, 3.5))
plt.plot(range(1, 10), inertias, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Inertia", fontsize=14)
plt.annotate('Elbow',
             xy=(4, inertias[3]),
             xytext=(0.55, 0.55),
             textcoords='figure fraction',
             fontsize=16,
             arrowprops=dict(facecolor='black', shrink=0.1)
            )
plt.axis([1, 8.5, 0, 1300])
save_fig("inertia_vs_k_plot")
plt.show()
```

▼ *** Saving figure inertia_vs_k_plot



Silueta original

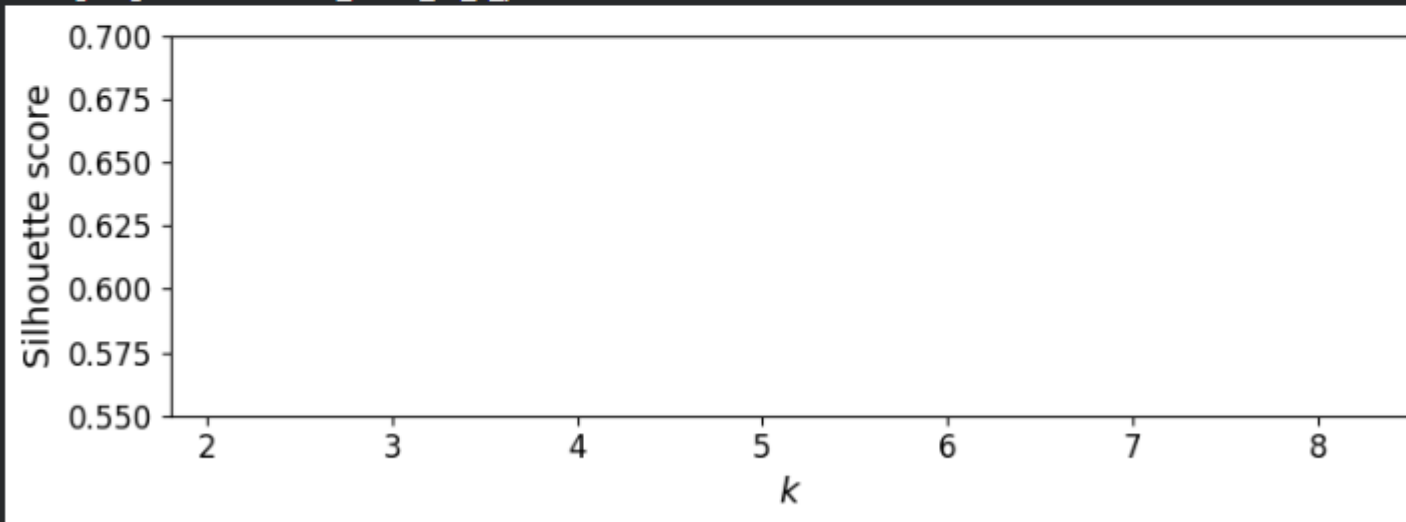


Silueta modificado

```
[415] ✓ 0s silhouette_scores = [silhouette_score(X, model.labels_)  
                      for model in kmeans_per_k[1:]]
```

```
[416] ✓ 0s ▶ plt.figure(figsize=(8, 3))  
plt.plot(range(2, 10), silhouette_scores, "bo-")  
plt.xlabel("$k$", fontsize=14)  
plt.ylabel("Silhouette score", fontsize=14)  
plt.axis([1.8, 8.5, 0.55, 0.7])  
save_fig("silhouette_score_vs_k_plot")  
plt.show()
```

... Saving figure silhouette_score_vs_k_plot



As you can see, this visualization is much richer than the previous one: in particular, although it confirms that $k = 4$ is the best choice, but it also underlines the fact that $k = 5$ is quite good as well.

Centros y std usados:

```

427] from sklearn.datasets import make_blobs
✓ 0 s

428] blob_centers = np.array(
    [[ 0.2, 1.3],
     [-1.5, 1.3],
     [-2.8, 1.8],
     [-3.8, 2.8],
     [-4.8, 1.3]])
    blob_std = np.array([0.1, 0.2, 0.1, 0.1, 0.1])

429] X, y = make_blobs(n_samples=2000, centers=blob_centers,
    cluster_std=blob_std, random_state=7)
✓ 0 s

```

- En los datos de Géron, ¿por qué el codo “prefiere” ($k = 4$) si `make_blobs` usó 5 centros?

La celda explica que el codo prefiere $k=4$ porque los dos blobs en la parte inferior izquierda están lo suficientemente cerca como para que K-Means los considere un solo clúster. Por lo tanto, añadir un quinto clúster para separarlos no reduce la inercia de manera significativa para formar un nuevo codo

- Con tus blobs separados, ¿el codo y la silueta coinciden en el mismo (k)? ¿Ese (k) es 5?

No, el método del codo y el coeficiente de silueta no coinciden exactamente en el mismo valor de k .

El método del codo sugiere que $k=4$ es una buena elección, ya que después de este punto, la reducción en la inercia no es tan significativa. Esto se debe a que los dos blobs de la parte inferior izquierda están muy cerca y K-Means tiende a agruparlos en un solo clúster.

Entonces, aunque el codo apunta a $k=4$, la silueta considera $k=5$ y también $k=4$ como opciones válidas, siendo $k=5$ un valor fuerte y más alineado con la configuración original de los datos.

- Si el codo sigue en 4, ¿qué te falta mover (distancia entre centros vs. blob_std)?

Si el método del codo sigue indicando $k=4$ y `make_blobs` usó 5 centros, significa que hay al menos dos de esos 5 centros que están demasiado cerca entre sí, o sus 'blobs' individuales son demasiado grandes, lo que hace que el algoritmo de K-Means los agrupe como uno solo. Para que el método del codo prefiera $k=5$, se tendría que aumentar la distancia entre los centros.

Disminuir el `blob_std` (la desviación estándar de los blobs): Si los blobs son más compactos, es más fácil para K-Means identificarlos como cúmulos separados. Reducir el `blob_std` haría que cada 'blob' sea más denso y, por lo tanto, menos propenso a mezclarse con un 'blob' vecino.

Captura de runtime:



Q Comandos + Código + Texto ▶ Ejecutar todo ▼

[423]
✓ 0 s

```
# Python ≥3.5 is required
import sys
assert sys.version_info >= (3, 5)

# Scikit-Learn ≥0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# Common imports
import numpy as np
import os

# to make this notebook's output stable across runs
np.random.seed(42)

# To plot pretty figures
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.rc('axes', labelsize=14)
mpl.rc('xtick', labelsize=12)
mpl.rc('ytick', labelsize=12)

# Where to save the figures
PROJECT_ROOT_DIR = "."
CHAPTER_ID = "unsupervised_learning"
IMAGES_PATH = os.path.join(PROJECT_ROOT_DIR, "images", CHAPTER_ID)
os.makedirs(IMAGES_PATH, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

▼ Clustering

Introduction – Classification vs Clustering

[424]
✓ 0 s

```
from sklearn.datasets import load_iris
```

[425]

```
data = load_iris()
```

